Student PIDs: Sparrow, ZLee

Available Expressions: The available expressions part of the assignment was mostly done; it just required setting up code for the transfer function and setting the out section for the while loop. To do this we used the transfer function Gen U (In – Kill), in code we can do this by setting the out equal to the in bitvector, then reseting the kill bits to fulfill the paraentheses part. Then finally if we OR that section, we get the full application of the transfer function.

Liveness: The program starts with assigning all used and defined variables in the universal set. From there, we organise the universal set to remove duplicates. Next, the order of traversal is assigned by starting with the entry block then adding each of its successors so long as it's not already in the order. All bitvectors are instantiated to empty sets the size of our universal set. For the sake of simplicity, a vector of type Value is also created for the use set and def set. Next, we begin populating the use and def vectors. It is important to make sure if a variable was defined before use, it should only be in the def set. If it is used before its definition, it is allowed to be in both sets. After the vectors are fully populated, they can be converted into bits to be added to the bitvectors. Now that our use and def sets are populated, we can calculate the in and out sets using our transfer functions. Starting with the last basic blocks, we calculate our out set as the union of the successors of the block. Our in set is the union of the use set and the difference between the out set and the kill set. We repeat the calculations of the in and out sets until we've reached a fixed point. Afterwards, all the sets are printed to show their values.

Reaching Definitions: This program defined things very similarly to the available expressions; the biggest difference is instead of doing expressions we are just using the raw instruction for doing all of our comparisons and computations. We do this by adding the universe to the run function that includes all instructions that have a return value or basically any instruction that defines a value as something. Then we get the order the same way and create the gen and kill sets by adding any instruction in a basic block to that blocks gen set and then add anytime that instruction is using a defined value to the kill set. Then we iterate through the entire section again doing so until we see the changed not be true or we get the same values twice in a row. In the actual loop we do the same thing for the previous out as the available expressions section. Then differently we have the in be the newly created meetUnion function that combines all the outs to be the in. Then we use the same transfer function again as GEN U (IN – KILL) to compute the new version of the out and end the loop if it doesn't change between this iteration and the out for the block previously. The other change from the original source code was creating a new printBitSet function for this one using instructions instead of expressions overloading the original function and having it use expressions instead of templates.